Machine Learning Tutorial: Vehicle Image Classification using CNN and Transfer Learning (MobileNetV2)

Student Name: Mahendar Kongolla **Student ID:** 24150793

GitHub Link: https://github.com/MahendarReddyKongolla/Individual-Assignment
Dataset Link: https://www.kaggle.com/datasets/mohamedmaher5/vehicle-classification

Introduction

Image classification is a fundamental task in computer vision, where the objective is to automatically assign labels to images based on their visual content. It plays a crucial role in real-world applications such as autonomous driving, traffic monitoring, and intelligent transportation systems, where accurate recognition of vehicles is essential for safety and decision-making.

With the rapid development of deep learning, Convolutional Neural Networks (CNNs) have become the preferred method for image classification tasks. These models automatically learn layered feature representations from raw images, identifying simple patterns such as edges and textures as well as more complex visual structures. However, building CNNs from scratch often requires large amounts of labelled data and substantial computational power, making it challenging when resources are limited.

To overcome these limitations, transfer learning has gained significant popularity. This approach utilises pre-trained models, such as MobileNetV2 which have already learned useful visual features from large-scale datasets like ImageNet. By reusing this prior knowledge, models can be adapted to new tasks more efficiently, leading to faster training and improved performance. In this tutorial, MobileNetV2 is first applied as a fixed feature extractor and then further refined through fine-tuning, where selected layers are retrained to better capture the characteristics of the vehicle classification dataset.

Dataset Overview

The dataset used in this tutorial consists of vehicle images organised into structured directories, where each folder corresponds to a specific vehicle category such as cars, buses, trucks, and other vehicle types. This folder-based organisation allows labels to be automatically assigned during data loading.

The dataset is sourced from Kaggle and accessed using KaggleHub, which ensures reproducibility and simplifies the process of downloading and managing the data. Such datasets typically contain variations in lighting conditions, backgrounds, and viewing angles, making them suitable for evaluating deep learning models in realistic scenarios.

Data Cleaning and Validation

Before training the model, it is essential to ensure that the dataset is clean and free from errors. A custom data cleaning function is implemented to improve dataset quality by:

- Removing unsupported file formats (e.g., non-image files)
- Detecting and deleting corrupted or unreadable images using image verification

This step is crucial because invalid data can: Cause runtime errors during training, Lead to incorrect feature learning and Negatively impact model performance.
By cleaning the dataset, we ensure that only valid and meaningful images are used for training.

Dataset Loading and Splitting

The cleaned dataset is loaded using TensorFlow's image_dataset_from_directory() function, which provides an efficient pipeline for handling image data.
The dataset is split into:

- 80% training set – used for learning patterns
- 20% validation set – used to evaluate model performance

Additional configurations include:

- Image resizing to 224 × 224 pixels, ensuring consistency and compatibility with deep learning models
- Batch size of 32, which balances memory usage and training speed
- Fixed random seed, ensuring reproducibility of results

This structured loading process enables efficient training and evaluation.

Data Preprocessing

To prepare the images for model training, preprocessing is applied to standardise input data.

- Pixel values are scaled from [0,255] to [0,1], which improves numerical stability and accelerates convergence
- Normalisation ensures that all inputs are within a consistent range making training more effective

This step is particularly important for neural networks as unscaled data can lead to unstable gradients and slower learning.

Data Pipeline Optimisation

To further enhance performance, TensorFlow data pipeline optimisations are applied:

- The map() function is used to efficiently apply preprocessing transformations to each batch
- The prefetch() function enables overlapping of data loading and model training, reducing idle time and improving overall efficiency

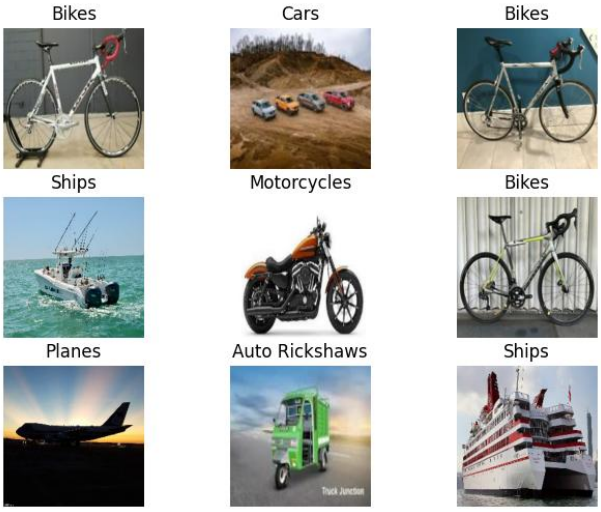
These optimisations ensure that the model trains faster and makes better use of available computational resources.

Exploratory Data Analysis (EDA)

In this tutorial, EDA primarily focuses on **visualising sample images** from the dataset. By displaying a subset of images from different classes, it is possible to observe:

- Variations in vehicle types and shapes
- Differences in background environments (e.g., roads, parking areas)
- Changes in lighting conditions and image quality
- Variability in object orientation and scale

The dataset is largely balanced, with most classes containing around 800 images. Minor variations are observed in classes such as cars and planes, but these differences are minimal and unlikely to affect model performance.



Baseline CNN Model

Overview of the Baseline Approach
A baseline Convolutional Neural Network (CNN) is implemented to establish a reference performance for the vehicle classification task. This model is trained from scratch and serves as a comparison point for more advanced techniques such as transfer learning.
The baseline model focuses on learning basic visual patterns directly from the dataset without using any pre-trained knowledge.

Model Architecture

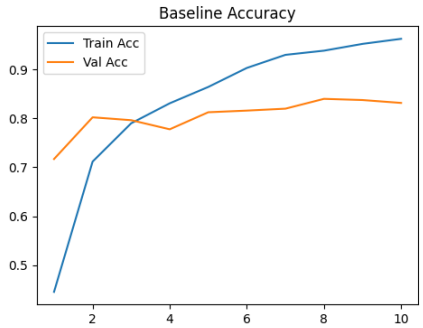
The CNN architecture is designed to progressively extract features from input images using multiple convolutional layers (32, 64, 128 filters) to capture edges, shapes, and textures. MaxPooling layers reduce spatial dimensions while preserving important information, improving efficiency. The extracted features are then processed by a fully connected dense layer (128 units) to learn complex patterns. A dropout layer (0.5) is applied to reduce overfitting, and a softmax output layer produces probability scores for each vehicle class. This simple architecture provides an effective baseline for image classification tasks.

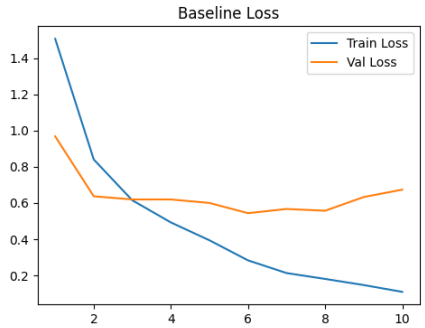
Training Setup

The model is trained using the Adam optimizer with sparse categorical crossentropy as the loss function for 10 epochs. All input images are resized to 224 × 224 pixels to ensure consistency. Additionally, invalid or corrupted images are skipped during training using TensorFlow's error-handling functionality, ensuring smooth and uninterrupted execution.

Training Performance

The training results show a steady improvement in performance, with training accuracy increasing rapidly from approximately 45% to 96%. In contrast, validation accuracy stabilises around 82–84%, indicating a gap between training and validation performance. The loss curves further support this observation, where training loss decreases consistently, while validation loss initially decreases but later begins to increase.



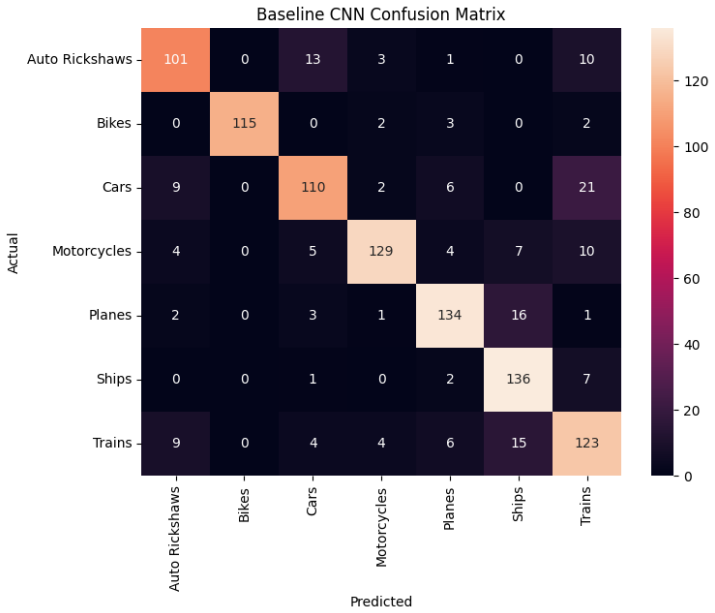


Evaluation

The confusion matrix provides deeper insights into model behaviour. The model performs well on classes such as ships, planes, and motorcycles, showing strong predictive accuracy. However, misclassifications are observed between visually similar classes, including cars and trains, auto rickshaws and cars, and planes and ships. These errors arise due to similarities in shape, structure, and background context, making it difficult for the baseline model to distinguish between certain vehicle types.

Classification Report Analysis

The baseline CNN achieves an overall accuracy of approximately **83%**, indicating a reasonable performance for a model trained from scratch. The macro and weighted averages for precision, recall, and F1-score are all around **0.83–0.84**, showing balanced performance across classes.



Performance Interpretation

The variation in class-wise performance suggests that the model struggles with classes that share similar visual characteristics. Vehicles such as cars, trains, and auto rickshaws often have overlapping features like shape and background context, leading to misclassification.

Observation

Although the baseline CNN achieves good overall accuracy, the inconsistency across classes highlights its limitation in learning fine-grained features. This reinforces the need for more advanced approaches such as transfer learning to improve classification performance.

Transfer Learning with MobileNetV2

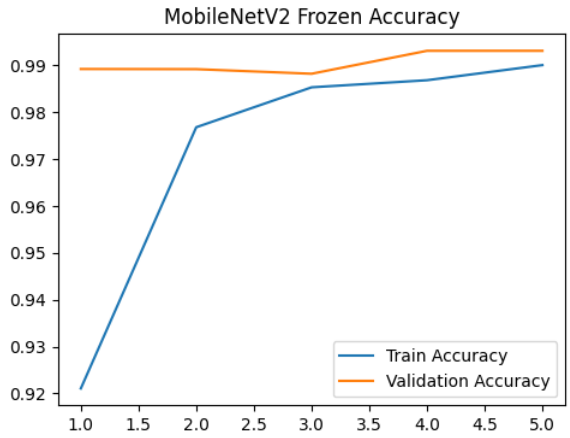
Introduction to Transfer Learning

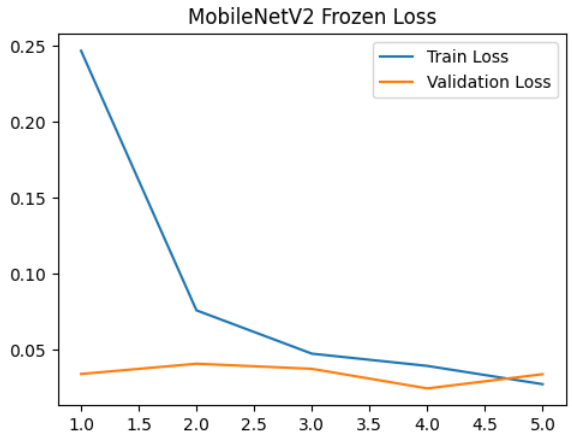
The limitations observed in the baseline CNN highlight the need for a more powerful approach capable of extracting richer and more discriminative features. To address this, transfer learning is applied using MobileNetV2, a pre-trained model trained on the ImageNet dataset. Instead of learning features from scratch, the model leverages previously learned representations, enabling faster training and improved performance.

Frozen MobileNetV2 Model

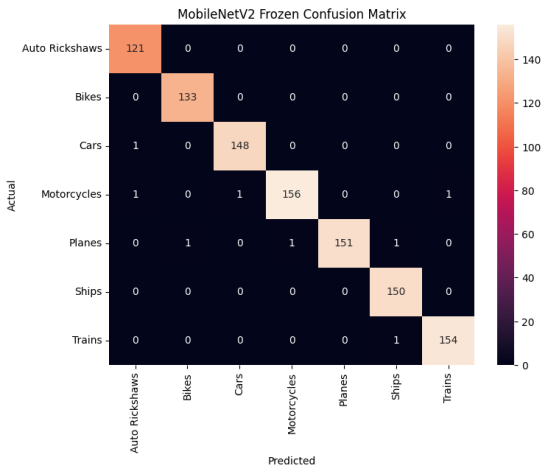
In this stage, MobileNetV2 is used as a fixed feature extractor by freezing all its pre-trained layers. A custom classification head is added on top of the base model, allowing the network to adapt to the vehicle classification task without modifying the learned weights.

The training results demonstrate a significant improvement compared to the baseline CNN. The model quickly achieves very high accuracy, with validation accuracy reaching approximately 99%. The training and validation curves remain closely aligned, indicating strong generalisation and no signs of overfitting. The loss curves further confirm this behaviour, showing a consistent decrease in both training and validation loss.



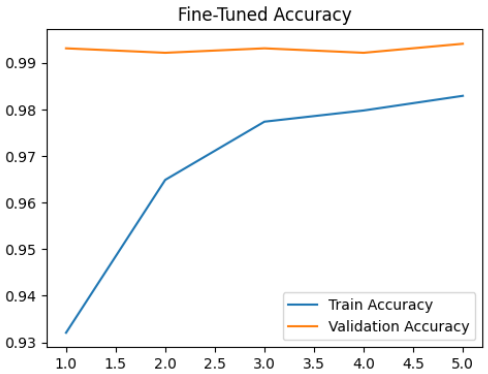


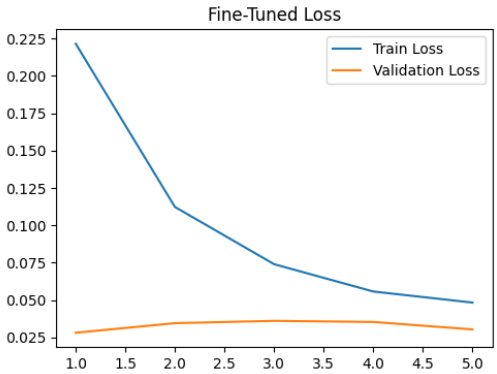
The classification report reflects near-perfect performance across all classes, with precision, recall, and F1-scores close to 1.00. The confusion matrix shows strong diagonal dominance, with almost all samples correctly classified. Only minimal misclassifications are observed, indicating that the model effectively distinguishes between different vehicle types.



Fine-Tuning MobileNetV2

To further enhance performance, fine-tuning is applied by unfreezing the upper layers of the MobileNetV2 model. This allows the network to adapt its high-level feature representations specifically to the vehicle dataset while retaining the general features learned from ImageNet. A lower learning rate is used to ensure stable updates and prevent large changes to the pre-trained weights.





The fine-tuned model maintains very high performance, with accuracy remaining around 99%. The training and validation accuracy curves show consistent improvement, while the loss continues to decrease steadily, indicating stable learning. Compared to the frozen model, fine-tuning results in slightly improved feature refinement and more confident predictions.

The classification report confirms this improvement, with all classes achieving precision, recall, and F1-scores close to 0.99 or higher. The confusion matrix further demonstrates near-perfect classification, with very few misclassifications. The model is now able to capture subtle differences between vehicle types more effectively.

Fine-Tuned Model Performance Analysis

The fine-tuned MobileNetV2 model achieves an overall accuracy of approximately **99%**, demonstrating excellent performance across all vehicle classes. Both macro and weighted averages for precision, recall, and F1-score are consistently around **0.99**, indicating that the model performs uniformly well without bias toward any particular class.

Class-wise performance shows that most categories achieve near-perfect precision and recall, particularly Bikes, Planes, and Ships, indicating strong feature learning. Motorcycles and Cars also perform very well, demonstrating accurate classification.

Minor variations are observed in Auto Rickshaws and Trains, where slightly lower recall suggests a few misclassifications due to visual similarities. Overall, fine-tuning improves the model's ability to capture subtle differences, resulting in strong generalisation and highly reliable performance.

Model Performance Comparison

A comparison of model accuracies clearly highlights the impact of transfer learning and fine-tuning. The baseline CNN achieves an accuracy of **83.2%**, which reflects moderate performance when trained from scratch. In contrast, the MobileNetV2 model with frozen layers significantly improves accuracy to **99.3%**, demonstrating the effectiveness of leveraging pre-trained features.

Further improvement is observed after fine-tuning, where the model achieves an accuracy of **99.4%**. Although the increase from the frozen model is relatively small, it indicates that fine-tuning helps the model adapt more precisely to dataset-specific features.

Overall, this comparison shows that transfer learning provides the largest performance gain, while fine-tuning offers additional refinement, resulting in the best overall performance.

Prediction Visualisation and Model Behaviour

Visualising Model Predictions

A prediction visualisation function is used to display sample images along with their predicted and ground truth labels. Correct predictions are shown in green, while incorrect ones are highlighted in red. This provides qualitative insight into model behaviour beyond numerical metrics.

Baseline CNN Predictions

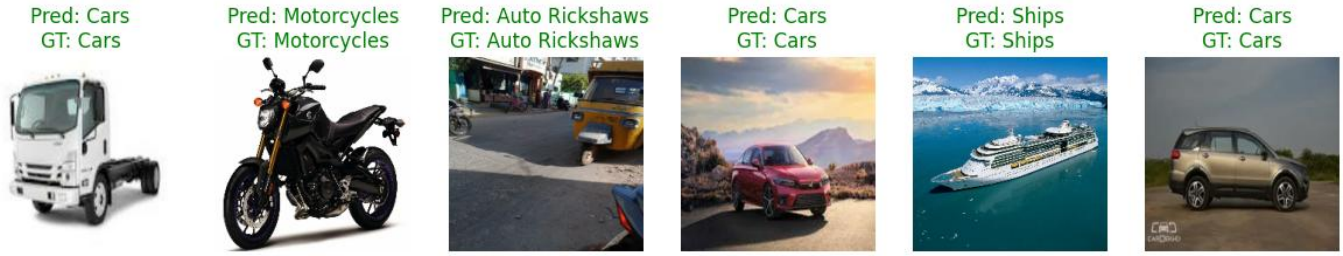
The baseline CNN shows several misclassifications, particularly between visually similar classes such as trains and auto rickshaws or planes and motorcycles. This indicates limited feature extraction and difficulty in capturing fine-grained differences.

Fig (a). Baseline CNN - Predictions

MobileNetV2 Frozen Model Predictions

The MobileNetV2 model significantly improves performance, with most predictions being correct. The model demonstrates better consistency and is able to distinguish between classes more effectively due to learned pre-trained features.

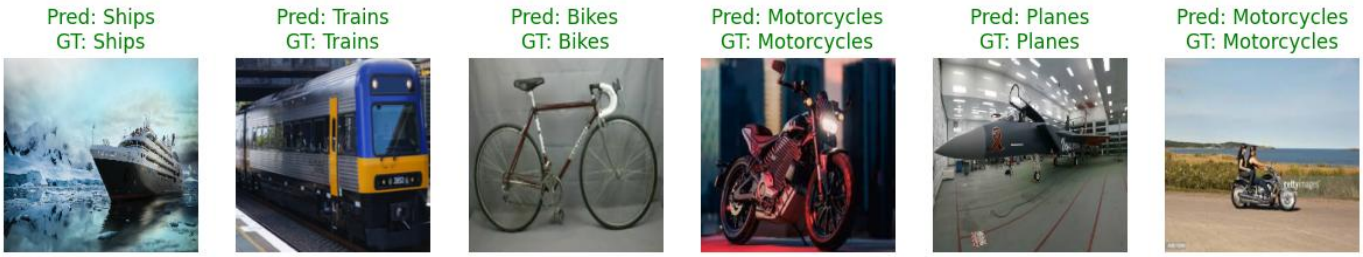
Fig (b). MobileNetV2 Frozen Model - Predictions



Fine-Tuned Model Predictions

The fine-tuned model achieves highly accurate and consistent predictions, with very few errors. It is better able to capture subtle differences between classes, resulting in improved classification performance even in challenging scenarios.

Fig (c). MobileNetV2 Fine Tuned Model - Predictions



Conclusion

This tutorial demonstrates the effectiveness of transfer learning for vehicle image classification. The baseline CNN achieved moderate accuracy (83.2%) with limitations in generalisation. In contrast, MobileNetV2 significantly improved performance, achieving 99.3% accuracy, while fine-tuning further

increased it to 99.4%. The results show that pre-trained models provide better feature extraction and more reliable predictions. Overall, transfer learning offers a powerful, efficient, and scalable solution for image classification, especially when data and computational resources are limited.

References

- TensorFlow (2024) *Transfer learning and fine-tuning tutorial*. Available at: https://www.tensorflow.org/tutorials/images/transfer_learning
- Kaggle (2024) *Vehicle Classification Dataset*. Available at: https://www.kaggle.com/datasets/mohamedmaher5/vehicle-classification
- Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H. (2017) *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. Available at: https://arxiv.org/abs/1704.04861
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.C. (2018) *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. Available at: https://arxiv.org/abs/1801.04381
- Pan, S.J. and Yang, Q. (2010) *A Survey on Transfer Learning*. IEEE Transactions on Knowledge and Data Engineering. Available at: https://ieeexplore.ieee.org/document/5288526